



**EMERITUS**

Learn. From the world's best.

# Probabilistic models for classification



# Naive Bayes' Algorithm

---

# Naive Bayes' Algorithm

Probabilistic classifier assuming feature independence

## Definition

A probabilistic classifier based on Bayes' Theorem, assuming feature independence

## Type

Simple yet effective classification algorithm, especially for text data

# Naive Bayes' Algorithm: Key Concepts

Understanding Bayes' Theorem in classification

$$P(A/B) = \frac{P(B/A) \cdot P(A)}{P(B)}$$

- **P(A|B):** Probability of class given data (Posterior)
- **P(B|A):** Probability of data given class (Likelihood)
- **P(A):** Initial probability of class (Prior)
- **P(B):** Overall probability of data (Evidence)

# Naive Bayes' Algorithm: Key Concepts

Core assumption and types of Naive Bayes



## Naive" Assumption

- Features are considered independent

## Types of Naive Bayes

- Gaussian: For continuous data (normal distribution)
- Multinomial: For count data like text
- Bernoulli: For binary or boolean data

# Example: Text Classification

Classifying messages as "spam" or "not spam"

Let's see a simple example of using Naive Bayes to classify messages as "spam" or "not spam"

## Step 1: Dataset

We have the following small dataset of labeled messages:

| Message                              | Label    |
|--------------------------------------|----------|
| "Win a free vacation now!"           | Spam     |
| "Hi, how are you?"                   | Not Spam |
| "Claim your free prize immediately!" | Spam     |
| "Let's catch up soon"                | Not Spam |
| "Exclusive offer just for you!"      | Spam     |

# Example: Text Classification

## Step 2 – Preprocessing the messages

Tokenisation: Split messages into individual words

Lowercasing: Convert all words to lowercase

Remove stopwords: Exclude common words like “a”, “the”, “is”

| Message                              | Label    | Tokens                                  |
|--------------------------------------|----------|-----------------------------------------|
| "Win a free vacation now!"           | Spam     | [win, free, vacation, now]              |
| "Hi, how are you?"                   | Not Spam | [hi, how, are, you]                     |
| "Claim your free prize immediately!" | Spam     | [claim, your, free, prize, immediately] |
| "Let's catch up soon"                | Not Spam | [let's, catch, up, soon]                |
| "Exclusive offer just for you!"      | Spam     | [exclusive, offer, just, for, you]      |



# Example: Text Classification

## Step 3 – Build the Vocabulary

Combine all unique tokens from the messages to create a vocabulary.

### **Vocabulary:**

{win, free, vacation, now, hi, how, are, you, claim, your, prize, immediately, let's, catch, up, soon, exclusive, offer, just, for}

# Example: Text Classification

## Step 4 – Calculate Probabilities

- **Prior Probabilities:**

$$P(\text{Spam}) = 3/5$$

$$P(\text{Not Spam}) = 2/5$$

- **Likelihoods:**

- For each word, calculate its probability given the class

- Example:

$$P(\text{free} | \text{Spam}) = (\text{Count of "free" in Spam} + 1) / (\text{Total Spam words} + \text{Vocabulary size})$$

- Repeat for all tokens in both classes

# Example: Text Classification

## Step 5 – Classify a New Message

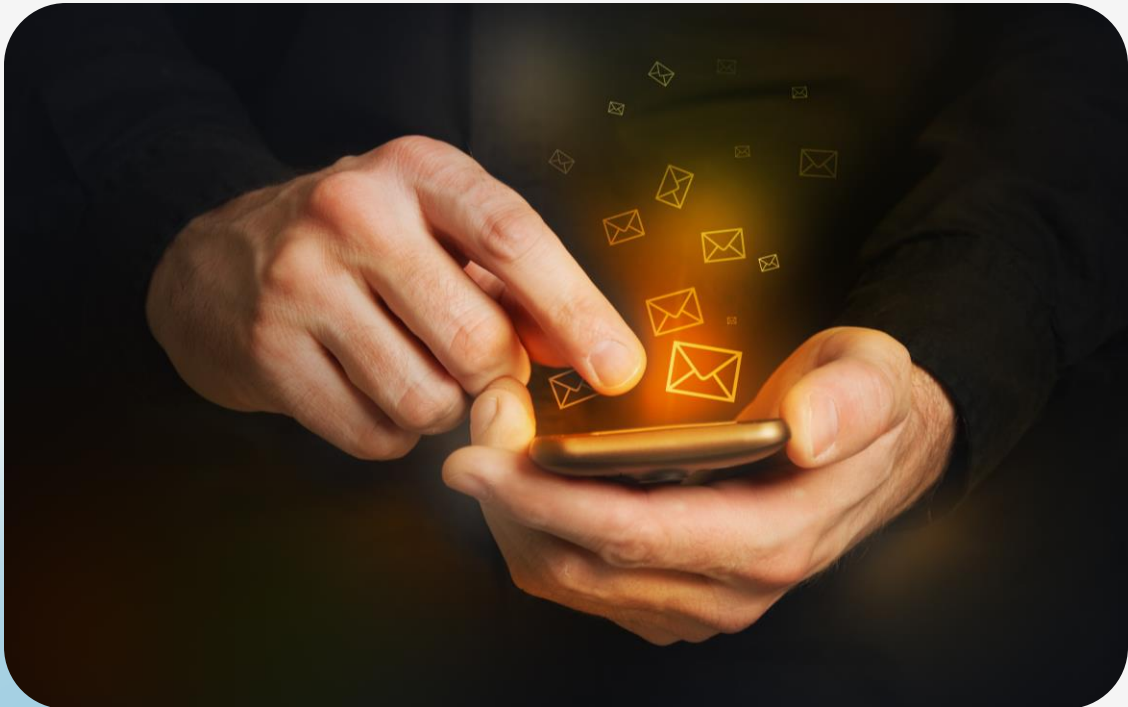
- New message: "Free prize for you!"
- Preprocessing: [free, prize, for, you]
- Calculate posterior probabilities using Bayes' Theorem

$$P(\text{Spam} \mid \text{message}) \propto P(\text{Spam}) \times P(\text{free} \mid \text{Spam}) \times P(\text{prize} \mid \text{Spam}) \times P(\text{for} \mid \text{Spam}) \times P(\text{you} \mid \text{Spam})$$

- Similarly, calculate for Not Spam

# Example: Text Classification

## Final Decision



- Compare posterior probabilities
- If  $P(\text{Spam} \mid \text{message}) > P(\text{Not Spam} \mid \text{message}) \rightarrow$  classify as Spam
- Otherwise  $\rightarrow$  classify as Not Spam

# Advantages

Why it's a popular choice



- **Fast and efficient:** Low computation, handles large datasets
- **Great for text data:** Used in spam filters, sentiment analysis
- **Performs well with small data:** Effective even with limited training samples

# Limitations

Key challenges to consider

## **Independence assumption**

May not reflect real-world data

## **Zero probability issue**

Can cause errors in prediction

## **Solution**

Use Laplace smoothing to avoid zero values

# TF-IDF Vectorisation

---

# TF-IDF Vectorization

Finding important words in documents

Stands for **Term Frequency–Inverse Document Frequency**

Measures how important a word is in a document

Importance is based on its frequency in one document vs many in the corpus



# TF-IDF Vectorisation

## Key Concepts

- **Term Frequency (TF):**

Shows how often a word appears in a document

Formula:

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

- **Inverse Document Frequency (IDF):**

Highlights how unique a word is across all documents

Formula:

$$IDF(t) = \log\left(\frac{\text{Total number of documents}}{\text{Number of documents containing term } t}\right)$$

# TF-IDF Vectorisation

## Key Concepts

- **TF-IDF Score:**

Combines TF and IDF to determine the importance of a term in a document

Formula:

$$TF - IDF(t, d) + TF(t, d) \times IDF(t)$$

# Applications

Where it's commonly used



- **Text classification:** Helps represent text for machine learning
- **Information retrieval:** Improves search results by ranking relevant documents

# Advantages and Limitations

Strengths and challenges

## Advantages

**Fewer features:** Focuses only on important words

**Good for sparse data:** Handles text with many zero values

## Limitations

**No context:** Doesn't understand word meaning or usage

**Fixed vocabulary:** Can't recognise new or unseen words

# Example: TF-IDF Calculation

## Step 1 – Prepare the dataset

- Walk through a basic TF-IDF example
- Use a small set of documents:
  - **Doc 1:** "The cat sat on the mat."
  - **Doc 2:** "The dog sat on the log."
  - **Doc 3:** "The cat chased the dog."

# Example: TF-IDF Calculation

## Step 2 – Preprocess the documents

- **Tokenisation:** Split text into words
- **Lowercasing:** Convert all words to lowercase
- **Remove Stopwords:** Exclude common words like “the”, “on”, “sat”

### Processed Tokens:

- **Doc 1:** [cat, sat, mat]
- **Doc 2:** [dog, log]
- **Doc 3:** [cat, chased, dog]

# Example: TF-IDF Calculation

## Step 3 – Calculate Term Frequency (TF)

| Term   | Doc 1 TF     | Doc 2 TF     | Doc 3 TF     |
|--------|--------------|--------------|--------------|
| cat    | $1/3 = 0.33$ | $0/3 = 0.00$ | $1/3 = 0.33$ |
| dog    | $0/3 = 0.00$ | $1/3 = 0.33$ | $1/3 = 0.33$ |
| sat    | $1/3 = 0.33$ | $0/3 = 0.00$ | $0/3 = 0.00$ |
| mat    | $1/3 = 0.33$ | $0/3 = 0.00$ | $0/3 = 0.00$ |
| log    | $0/3 = 0.00$ | $1/3 = 0.33$ | $0/3 = 0.00$ |
| chased | $0/3 = 0.00$ | $0/3 = 0.00$ | $1/3 = 0.33$ |

- Count how often each word appears in a document
- $TF = (\text{Term count}) / (\text{Total terms in the document})$
- Do this for every unique word in each document

# Example: TF-IDF Calculation

## Step 4 – Calculate Inverse Document Frequency (IDF)

Calculate the inverse document frequency for each term. IDF is calculated as:

$$IDF(t) = \text{Log} \left( \frac{N}{n_t} \right)$$

where  $N$  is the total number of documents and  $n_t$  is the number of documents containing the term  $t$ .



# Example: TF-IDF Calculation

## Calculation

Total documents  $N=3$

| Term   | Documents Containing Term | IDF                                          |
|--------|---------------------------|----------------------------------------------|
| cat    | 2                         | $\log\left(\frac{3}{2}\right) \approx 0.176$ |
| dog    | 2                         | $\log\left(\frac{3}{2}\right) \approx 0.176$ |
| sat    | 2                         | $\log\left(\frac{3}{2}\right) \approx 0.176$ |
| mat    | 1                         | $\log\left(\frac{3}{1}\right) \approx 0.477$ |
| log    | 1                         | $\log\left(\frac{3}{1}\right) \approx 0.477$ |
| chased | 1                         | $\log\left(\frac{3}{1}\right) \approx 0.477$ |

# Example: TF-IDF Calculation

## Step 5: Calculate TF-IDF

Multiply the TF and IDF values for each term in each document

| Term   | Doc 1 TF-IDF                      | Doc 2 TF-IDF                      | Doc 3 TF-IDF                      |
|--------|-----------------------------------|-----------------------------------|-----------------------------------|
| cat    | $0.33 \times 0.176 \approx 0.058$ | $0.00 \times 0.176 = 0.00$        | $0.33 \times 0.176 \approx 0.058$ |
| dog    | $0.00 \times 0.176 = 0.00$        | $0.33 \times 0.176 \approx 0.058$ | $0.33 \times 0.176 \approx 0.058$ |
| sat    | $0.33 \times 0.176 \approx 0.058$ | $0.00 \times 0.176 = 0.00$        | $0.00 \times 0.176 = 0.00$        |
| mat    | $0.33 \times 0.477 \approx 0.157$ | $0.00 \times 0.477 = 0.00$        | $0.00 \times 0.477 = 0.00$        |
| log    | $0.00 \times 0.477 = 0.00$        | $0.33 \times 0.477 \approx 0.157$ | $0.00 \times 0.477 = 0.00$        |
| chased | $0.00 \times 0.477 = 0.00$        | $0.00 \times 0.477 = 0.00$        | $0.33 \times 0.477 \approx 0.157$ |

# TF-IDF Implementation

---

# TF-IDF Implementation

Step-by-step in Python using Scikit-learn

```
## Importing Libraries  
import pandas as pd  
from sklearn.feature_extraction.text import TfidfVectorizer  
  
#Load the Data from CSV with specified encoding  
df pd.read_csv(spam.csv", encoding='ISO-8859-1').iloc[:,0:2]  
  
## Check the DataFrame  
print("DataFrame Head:")  
df.head()
```

# TF-IDF Implementation

Step-by-step in Python using Scikit-learn

**Output:**

Out[13]:

|   | v1   | v2                                                |
|---|------|---------------------------------------------------|
| 0 | ham  | Go until jurong point, crazy.. Available only ... |
| 1 | ham  | Ok lar... Joking wif u oni...                     |
| 2 | spam | Free entry in 2 a wkly comp to win FA Cup fina... |
| 3 | ham  | U dun say so early hor... U c already then say... |
| 4 | ham  | Nah I don't think he goes to usf, he lives aro... |

# TF-IDF Implementation

```
## Applying TF-IDF Vectorizer
#Initialize the TF-IDF vectorizer, removing stopwords and numbers
tfidf_vectorizer TfidfVectorizer(stop_words='english',
                                token_pattern=r'\b[a-zA-Z]+\b') # Only keep alphabetic words

# Fit and transform the message column (v2) to create the TF-IDF matrix
tfidf_matrix = tfidf_vectorizer.fit_transform(df['v2'])

#Convert the TF-IDF matrix to a DataFrame for better visualization
tfidf_df pd.DataFrame(tfidf_matrix.toarray(),
                      columns=tfidf_vectorizer.get_feature_names_out())

#Display the TF-IDF DataFrame
print("\nTF-IDF DataFrame:")
tfidf_df
```

# TF-IDF Implementation

## Output:

Out[14]:

[illegible]

# TF-IDF Implementation

```
## Displaying the feature names  
print("\nFeature names (words):")  
print(tfidf_vectorizer.get_feature_names_out())
```

## Output:

```
Feature names (words):  
['aa' 'aah' 'aaniye' ... 'zoom' 'zouk' 'zyada']
```



# Naïve Bayes' Implementation

---

# Naïve Bayes' Implementation

## Importing Libraries

- NumPy and Pandas are used for numerical operations and data manipulation.
- Matplotlib and Seaborn are used for visualization.
- sklearn provides tools for machine learning, including model selection and evaluation metrics.

```
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
from sklearn.model_selection import train_test_split  
from sklearn.feature_extraction.text import TfidfVectorizer  
from sklearn.naive_bayes import MultinomialNB  
from sklearn.metrics import classification_report, confusion_matrix
```

# Naïve Bayes' Implementation

## Loading the Data

- The dataset contains SMS messages classified as 'ham' (not spam) or 'spam'.
- The target variable indicates the message type.
- Load the dataset

```
df = pd.read_csv('path_to_your_file.csv', encoding='ISO--8859-1').iloc[:,0:2]
```

```
# Data overview
```

```
print("DataFrame Head:")
```

```
print(df.head())
```

```
print("\nDataFrame Info:")
```

```
print(df.info())
```

# Naïve Bayes' Implementation

## Loading the Data

### Output:

DataFrame Head:

|   | v1   | v2                                                |
|---|------|---------------------------------------------------|
| 0 | ham  | Go until jurong point, crazy.. Available only ... |
| 1 | ham  | Ok lar... Joking wif u oni...                     |
| 2 | spam | Free entry in 2 a wkly comp to win FA Cup fina... |
| 3 | ham  | U dun say so early hor... U c already then say... |
| 4 | ham  | Nah I don't think he goes to usf, he lives aro... |

# Naïve Bayes' Implementation

## Data Preprocessing

```
## Applying TF-IDF Vectorizer  
#Initialize the TF-IDF vectorizer, removing stopwords and numbers  
tfidf_vectorizer TfidfVectorizer(stop_words='english',  
                                token_pattern=r'\b[a-zA-Z]+\b') # Only keep alphabetic words  
  
# Fit and transform the message column (v2) to create the TF-IDF matrix  
tfidf_matrix tfidf_vectorizer.fit_transform(df['v2'])  
  
#Convert the TF-IDF matrix to a DataFrame for better visualization  
tfidf_df pd.DataFrame(tfidf_matrix.toarray(), columns=tfidf_vectorizer.get_feature_names_out())  
  
# Display the TF-IDF DataFrame  
print("\nTF-IDF DataFrame:")  
tfidf_df
```

# Data Preprocessing

[illegible]

# Split the Data

## Prepare training and testing sets

- The `train_test_split` function splits the dataset into training (80%) and testing (20%) subsets
- Features and target

```
X = tfidf_matrix  
y df[ 'v1' ].map({ 'ham' : 0, 'spam' : 1}) # Encoding Labels as 0 for ham and 1 for spam  
  
#Split the data into training and testing sets  
X_train, X_test, y_train, y_test train_test_split(X, y, test_size=0.2, random_state=42)
```

# Naïve Bayes' Implementation

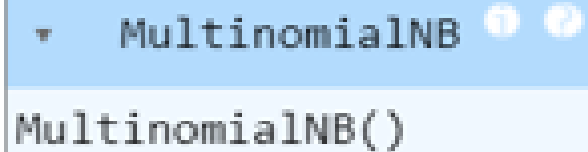
## Train and Test the Naive Bayes Model

- Here, we create an instance of the Multinomial Naive Bayes classifier and fit it to the training data.
- Create a Multinomial Naive Bayes classifier

```
nb_model = MultinomialNB()  
nb_model.fit(X_train, y_train)
```

### Output:

Out[9]:



A screenshot of a Jupyter Notebook output cell. It shows a blue header bar with a dropdown arrow, the text 'MultinomialNB', and two circular icons. Below the header, the text 'MultinomialNB()' is displayed in a light blue box.

```
MultinomialNB()
```



# Naïve Bayes' Implementation

## Train and Test the Naive Bayes Model

- The predict method generates predictions for the test data based on the model trained in the previous step.

```
y_train_prednb_model.predict(X_train)  
y_test_pred nb_model.predict(X_test)
```

# Naïve Bayes' Implementation

## Evaluate the Model

- The confusion matrix shows the number of true vs. predicted classifications.
- The classification report includes precision, recall, and F1-score.
- Evaluate the model on training data

```
print("Confusion Matrix: Train Data")
print(confusion_matrix(y_train, y_train_pred))
print("\nClassification Report (Train Data): ")
print(classification_report(y_train, y_train_pred))
print("=====")
```

```
#Evaluate the model on test data
print("Confusion Matrix: Test Data")
print(confusion_matrix(y_test, y_test_pred))
print("\nClassification Report (Test Data): ")
print(classification_report(y_test, y_test_pred))
```

# Naïve Bayes' Implementation

Confusion Matrix: Train Data

```
[[3860  0]
 [ 104 493]]
```

Classification Report (Train Data):

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.97      | 1.00   | 0.99     | 3860    |
| 1            | 1.00      | 0.83   | 0.90     | 597     |
| accuracy     |           |        | 0.98     | 4457    |
| macro avg    | 0.99      | 0.91   | 0.95     | 4457    |
| weighted avg | 0.98      | 0.98   | 0.98     | 4457    |

=====

Confusion Matrix: Test Data

```
[[965  0]
 [ 41 109]]
```

Classification Report (Test Data):

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.96      | 1.00   | 0.98     | 965     |
| 1            | 1.00      | 0.73   | 0.84     | 150     |
| accuracy     |           |        | 0.96     | 1115    |
| macro avg    | 0.98      | 0.86   | 0.91     | 1115    |
| weighted avg | 0.96      | 0.96   | 0.96     | 1115    |

# Naïve Bayes' Implementation

## Train and Test the Naive Bayes Model

```
new_data = pd.DataFrame ([["How are you doing? Did you finish your work?"], ['Nice weather! Lets go for a walk.'], ['Free! Free! Free!']])
new_data
```

### Output:

Out[19]:

|   | v2                                           |
|---|----------------------------------------------|
| 0 | How are you doing? Did you finish your work? |
| 1 | Nice weather! Lets go for a walk.            |
| 2 | Free! Free! Absolutely free!                 |

```
new_data_vector = tfidf_vectorizer.transform(new_data['v2'])
nb_model.predict(new_data_vector)
```

### Output:

Out[21]: array([0, 0, 0], dtype=int64)

# Q&A Session



Open floor for questions and clarifications.